Security Audit — logipay-portal

Дата: 14 октября 2026 г.
Репозиторий: logipay-portal (ветка `main`)
Аудитор: автоматизированный обзор кода
Формат: `Риск` → `Доказательства (файл:строка)` → `Сценарий` → `Фикс`

---

1. Scope (что проверено)

- Конфигурация проекта: `package.json`, `next.config.ts`, `vercel.json`
- Аутентификация/сессии: `lib/auth.ts`
- API-поверхность: `app/api/**` (паттерн-скан + выборочная проверка чувствительных роутов)
- Паттерны риска: RCE/XSS, SQL-инъекции, секреты, cookie/CORS/CSRF, авторизация/IDOR

---

2. Итоговая оценка

Область	Статус
Хранение паролей (PBKDF2)	✅ Хорошо
Cookie-флаги сессии	✅ Хорошо
Параметризация SQL	✅ В основном хорошо
RCE/XSS индикаторы	✅ Не найдено в runtime
Авторизация/IDOR на `[id]`	⚠️ Требует верификации (P1)
Защита ingest-эндпоинтов	⚠️ Требует усиления (P1)
Security headers	⚠️ Отсутствуют (P2)
Минимизация audit-логов	⚠️ Улучшить (P2)
CVE-аудит зависимостей	⏳ Не выполнен (нужен прогон)

---

3. Что сделано хорошо (подтверждено кодом)

`lib/auth.ts`:
- Cookie сессии: `HttpOnly`, `SameSite=Strict`, `Secure` (на HTTPS)
  - `lib/auth.ts:124` — `sessionCookie()`
  - `lib/auth.ts:128` — `clearSessionCookie()`
- Сессии хранятся как **хеш токена** (`token_hash`), не в plaintext — `createSession()` / `getPortalUser()`
- Пароли: **PBKDF2-SHA256**, 210 000 итераций — `hashPassword()`
- Проверка пароля в **constant-time** стиле (XOR-накопление различий) — `verifyPassword()`
- Минимум итераций валидируется (`< 100_000` → отказ)

SQL: в API преимущественно `prepare().bind()` → низкий риск SQL-инъекций.

RCE/XSS: конструкций `eval` / `new Function` / `dangerouslySetInnerHTML` в runtime-коде не найдено. `execSync` встречается только в локальном скрипте `scripts/reset-local-passwords.mjs`.

---

4. Находки по приоритету

P1 — Возможные ошибки авторизации/IDOR на ресурсах с `[id]`

Доказательства (поверхность):

- `app/api/documents/[id]/route.ts:9`
- `app/api/trip-documents/[id]/route.ts:9`
- `app/api/vehicle-files/[id]/route.ts:8`
- `app/api/payroll/[id]/pdf/route.ts:10`
- `app/api/staff/[id]/export/route.ts:9`
- `app/api/trip-chat/[id]/attachment/route.ts:10`

Сценарий: авторизованный пользователь (например, driver) подставляет чужой `id` и получает чужие документы/выплаты, если проверка принадлежности (ownership) не строгая.

Фикс:
1. Единый guard-хелпер `assertCanAccess(user, resourceOwner, action)` с принципом **deny-by-default**.
2. На каждом `[id]`-роуте — проверка роли **и** принадлежности записи (`record.user_id === user.id` или явное право по роли).
3. Негативные тесты: чужой `id` → `403/404`.

P1 — Ingest-эндпоинты требуют строгой периметровой защиты

Доказательства:
- `app/api/tracking/ingest/route.ts:49`
- `app/api/tachograph/ingest/route.ts:30`
- `app/api/tracking/fleetboard/ingest/route.ts:91`

Сценарий: подмена/флуд телеметрии и тахограф-данных, если нет строгой проверки источника.

Фикс:
1. Обязательный токен в `Authorization` **или** HMAC-подпись payload (`TRACKER_INGEST_TOKEN` уже упоминается в `VERCEL_DEPLOY.md`).
2. Anti-replay: окно по timestamp + nonce.
3. Rate-limit по ключу интеграции/IP.
4. Явный `401/403` при пустом/невалидном токене.

P2 — Избыточные данные в `audit_log`

Доказательства: множественные `JSON.stringify({...})` в `audit_log.details`, напр.:
- `app/api/employee-documents/route.ts:28`
- `app/api/trip-documents/route.ts:28`
- `app/api/direct-chat/route.ts:59`

Сценарий: накопление PII/метаданных файлов дольше необходимого.

Фикс:
1. Минимизировать поля (не писать содержимое пользовательских данных).
2. Retention-политика (напр. 90–180 дней).
3. Маскирование чувствительных значений.

P2 — Нет security-headers политики

Доказательства: `next.config.ts` фактически пустой.

Сценарий: ослабленный baseline против clickjacking / MIME-sniffing.

Фикс: добавить заголовки через `headers()` в `next.config.ts`:
- `Content-Security-Policy` (базовая, поэтапное ужесточение)
- `X-Frame-Options: DENY`
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `Permissions-Policy` (минимально необходимая)

P3 — `execSync` в операционном скрипте

Доказательства: `scripts/reset-local-passwords.mjs:7` — `import { execSync } from "node:child_process"`.

Сценарий: низкий риск (локальный admin-скрипт), но опасно при передаче непроверенного ввода в shell.

Фикс: фиксированные команды/аргументы; не принимать свободный пользовательский ввод в shell-интерполяцию.

---

5. Зависимости (CVE)

CVE-аудит пакетным менеджером **не запускался** в этой сессии. Рекомендуется прогнать и зафиксировать в CI:

```bash
npm audit --audit-level=high
```

---

6. Quality gates (не валидированы прогоном в этой сессии)

Gate	Статус
Build	⏳ не проверено
Lint	⏳ не проверено
Typecheck	⏳ не проверено
Tests	⏳ не проверено

Команды для проверки:

```bash
npm run lint
npm run build
npm test
```

---

7. План исправлений (порядок)

1. **P1:** унифицировать authz-guards для всех `[id]`-роутов + тесты на IDOR.
2. **P1:** защитить ingest-эндпоинты токеном/HMAC + rate-limit + anti-replay.
3. **P2:** security headers в `next.config.ts`.
4. **P2:** минимизация и retention для `audit_log`.
5. **Регулярно:** `npm audit` в CI.